Module 5 — Python fundamentals and coding interview problem solving

Learning outcomes

Write small correct programs using core data structures, functions, loops, exceptions and tests. Explain time/space complexity.

1. Data structures and when to use them

• List: ordered, mutable sequence; index access O(1), membership O(n).

• Tuple: ordered immutable sequence.

• Set: unique values; average membership O(1).

• Dict: key-value mapping; average lookup/update O(1).

Choose based on required operations, not familiarity.

2. Mutability and identity

Mutable objects can change in place. == checks value equality; `is` checks object identity. Use `is None` for None checks. Avoid mutable default arguments.

3. Functions

Use descriptive names, small units of work, explicit return values and validated inputs. `*args` collects positional extras; `**kwargs` collects keyword extras.

4. Exceptions

Catch specific expected errors. Do not use bare `except` to hide bugs. Use context managers (`with open(...)`) for resources. Validate at system boundaries.

5. Comprehensions, sorting and standard library

Know `enumerate`, `zip`, `sorted(key=...)`, `collections.Counter`, `defaultdict`, `deque`, `heapq`, `datetime`, `csv`, `json`, `pathlib`.

6. Complexity basics

O(1), O(log n), O(n), O(n log n), O(n²). Explain both time and extra space. A nested loop is not automatically O(n²) if its bounds differ; analyse actual work.

Worked examples

Frequency count

```
from collections import Counter
def frequencies(values):
    return Counter(values)
```

Two-sum

```
def two_sum(nums, target):
    seen = {}
    for i, value in enumerate(nums):
        needed = target - value
        if needed in seen:
            return [seen[needed], i]
        seen[value] = i
    return None
```

Average O(n) time and O(n) space. Define whether returning one pair or all pairs is required.

Group records

```
from collections import defaultdict
def group_by_city(rows):
    grouped = defaultdict(list)
    for row in rows:
        grouped[row["city"]].append(row)
    return dict(grouped)
```

Guided exercises with solutions

1. Reverse words in a sentence while preserving word order reversal.

```
def reverse_words(sentence):
    return " ".join(sentence.split()[::-1])
```

2. Return common elements without duplicates.

```
def common_unique(a, b):
    return set(a) & set(b)
```

If output order matters, preserve the order from a:

```
def common_in_a_order(a, b):
    bset = set(b)
    seen = set()
    result = []
    for item in a:
        if item in bset and item not in seen:
            result.append(item)
            seen.add(item)
    return result
```

3. Find the most frequent item; on ties choose lexicographically smallest.

```
from collections import Counter
def most_frequent(values):
    if not values:
        return None
    counts = Counter(values)
    best_count = max(counts.values())
    return min(x for x, c in counts.items() if c == best_count)
```

4. Merge two sorted lists.

```
def merge_sorted(a, b):
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out + a[i:] + b[j:]
```

5. Validate balanced brackets.

```
def balanced(text):
    pairs = {')': '(', ']': '[', '}': '{'}
    stack = []
    for ch in text:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack
```

Independent exercises

6. Return the first repeated integer in scan order.

7. Rotate a list right by k positions.

8. Check whether two strings are anagrams, ignoring spaces and case.

9. Find the longest word in a sentence; define tie behaviour.

10. Implement a simple LRU cache conceptually; use `OrderedDict` for an implementation.

11. Find the maximum subarray sum (Kadane's algorithm).

12. Generate Fibonacci values lazily with a generator.

Solutions

```
# 6
def first_repeated(values):
    seen = set()
    for x in values:
        if x in seen:
            return x
        seen.add(x)
    return None

# 7
def rotate_right(values, k):
    if not values:
        return values
    k %= len(values)
    return values[-k:] + values[:-k] if k else values[:]

# 8
def are_anagrams(a, b):
    clean = lambda s: sorted(ch.lower() for ch in s if not ch.isspace())
    return clean(a) == clean(b)
```

```
# 9: first longest word wins
def longest_word(sentence):
    words = sentence.split()
    return max(words, key=len) if words else None

# 11
def max_subarray_sum(nums):
    if not nums:
        return None
    best = current = nums[0]
    for x in nums[1:]:
        current = max(x, current + x)
        best = max(best, current)
    return best

# 12
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b
```

For 10, the key design is a hash map plus doubly linked list (O(1) get/put); Python's `OrderedDict` offers a standard implementation.

Testing checklist

For every function test empty input, one element, duplicates, negative values, unexpected types, boundary k, ties and large input as appropriate.

Mini mock (35 minutes, 30 marks)

• 8 marks: implement a frequency-based function.

• 8 marks: solve a two-pointer problem.

• 6 marks: parse and validate JSON records.

• 4 marks: explain complexity.

• 4 marks: write at least four tests.

External practice for this module

1. HackerRank Python — complete Introduction, Basic Data Types, Strings, Sets, Collections, Functions and Exceptions. Aim for at least 20 problems.

2. Exercism Python track — complete 10 beginner exercises and read the mentor/community discussion after your own attempt.

3. LeetCode — Array and String problems — solve 8 Easy problems after learning lists, loops and functions.

4. LeetCode — Hash Table problems — solve 8 Easy problems after dictionaries and sets; then try 2 Medium problems.

5. NeetCode Roadmap — use the Arrays & Hashing and Two Pointers sections as a topic checklist, not as a race to finish every problem.

Exit requirement: solve 15 beginner/easy problems and 5 interview-style problems without copying. For every solution, state time and space complexity and test empty input, duplicates and boundary cases.